

Programowanie aplikacji WWW w technologii .NET

Wprowadzenie do platformy ASP .NET Core

Ireneusz Mrozek

1

Informacje kontaktowe

Ireneusz Mrozek
e-mail: i.mrozek@pb.edu.pl
pok. A117 (118)

2

Wykład:

1. Sprawdzian w ostatnim lub przedostatnim tygodniu zajęć (przed sesją).
2. Sprawdzian poprawkowy (przed sesją lub w pierwszym tygodniu sesji)

Pracownia specjalistyczna:

Zasady przedstawi prowadzący.

Ocena z wykładu:

Punkty ze sprawdzianu[%]	Ocena
0 - 50)	2.0
<50 - 60)	3.0
<60 - 70)	3.5
<70 - 80)	4.0
<80 - 90)	4.5
<90 - 100)	5.0

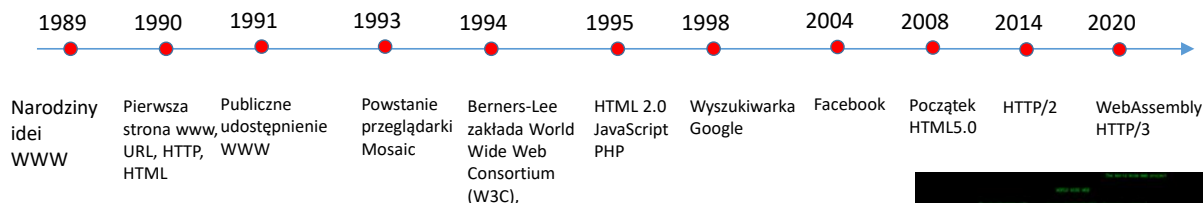
3

Materiały (wykład + pracownia specjalistyczna) umieszczane będą na platformie CEZ (cez.wi.pb.edu.pl)

KURS: Programowanie aplikacji WWW w technologii .NET (st Inf)

4

Historia WWW



1990 - Tim Berners-Lee opublikował pierwszą stronę WWW (sierpień 1991 roku). Połączył on istniejącą komunikację internetową z hipertekstem, który w tamtych czasach ograniczony był do przeglądania informacji na pojedynczym komputerze.



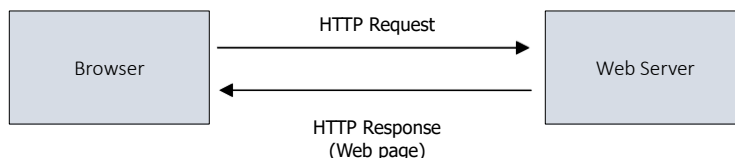
1993 - Premiera Mosaic, pierwszej przeglądarki internetowej obsługującej grafikę. Mosaic pomaga w popularyzacji WWW i staje się podstawą dla późniejszych przeglądarek.



5

Idea serwisów WWW

Serwis WWW (ang. World Wide Web) to zbiór stron internetowych dostępnych dla użytkowników za pomocą przeglądarek internetowych. Serwis WWW składa się z połączonych hiperłączami dokumentów napisanych głównie w języku HTML, które są interpretowane przez przeglądarki internetowe. WWW połączyło ideę hipertekstu i internetu, umożliwiając: łatwy dostęp do informacji, szybką komunikację, wymianę wiedzy na skale światowej.



HTTP:

protokół komunikacyjny używany do przesyłania dokumentów HTML między serwerem a przeglądarką internetową. Przeglądarka wysyła do serwera żądanie HTTP (GET/POST), aby pobrać stronę internetową. Serwer zwraca odpowiedź, która zawiera zasoby takie jak pliki HTML, CSS, JavaScript, obrazy.

HTML:

język znaczników, który opisuje strukturę treści stron internetowych. Każda strona WWW to dokument HTML. HTML jest stale rozwijany, obecnie używamy HTML5, który wspiera bogate treści multimedialne i nowoczesne aplikacje webowe.

6

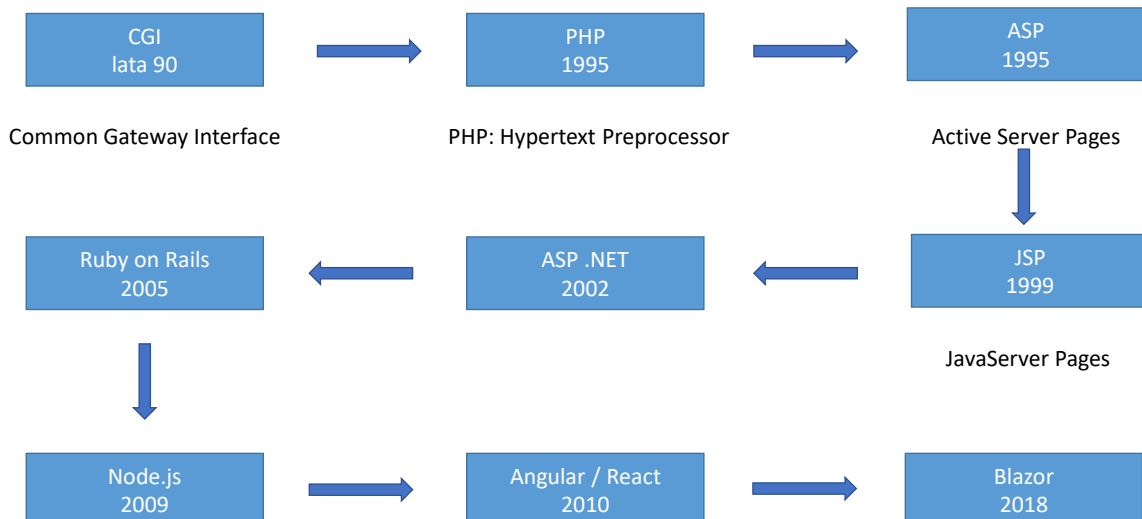
Kluczowe technologie oprócz HTML

CSS (Cascading Style Sheets) – język używany do definiowania wyglądu i stylu stron WWW, który pozwala na precyzyjne formatowanie elementów strony, takich jak kolory, układ, czcionki czy odstępy. CSS oddziela warstwę prezentacyjną od treści, co ułatwia utrzymanie i modyfikację wyglądu stron.

JavaScript – język skryptowy, który pozwala na dodanie interaktywności i dynamicznych zmian na stronie, takich jak obsługa kliknięć, animacje, walidacja formularzy czy asynchroniczne ładowanie danych. JavaScript działa bezpośrednio w przeglądarce, umożliwiając tworzenie nowoczesnych, interaktywnych aplikacji webowych.

7

Ewolucja technologii tworzenia aplikacji internetowych



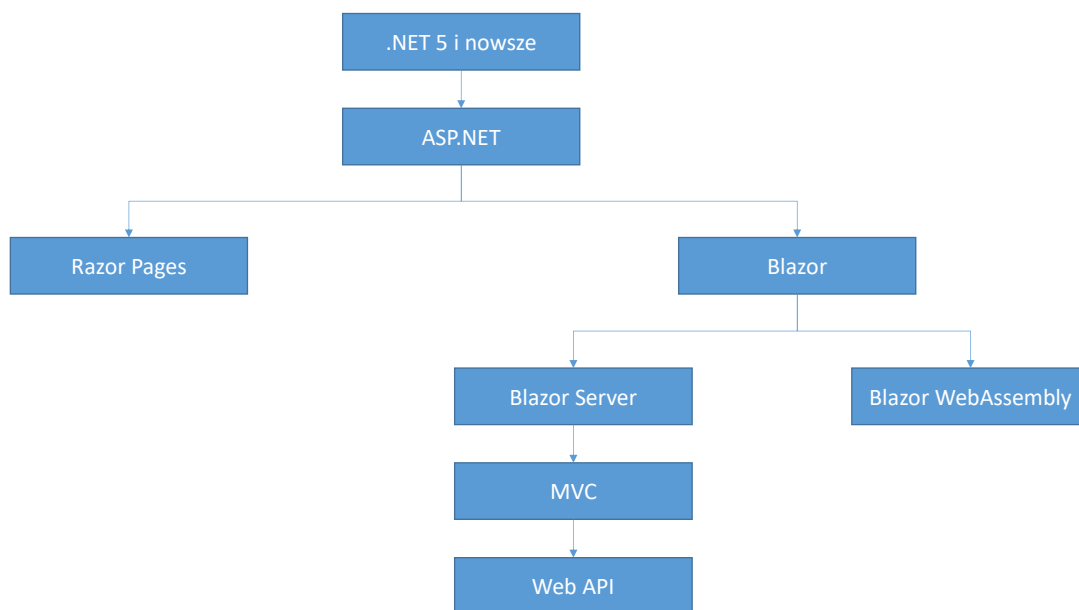
8

Rozwój technologii Microsoft tworzenia aplikacji WWW

Rok	Wydarzenie
1996	Premiera ASP (Active Server Pages) - technologii serwerowej do dynamicznego generowania stron WWW
2002	ASP.NET 1.0 - Wprowadzenie nowej architektury, opartej na modelu programowania zdarzeniowego oraz Web Forms.
2005	ASP.NET 2.0 - Dodanie funkcji takich jak Membership, Role Management i nowe kontrolki.
2008	ASP.NET 3.5 - Wprowadzenie LINQ, AJAX oraz wsparcie dla nowego modelu programowania w aplikacjach webowych.
2012	ASP.NET 4.5 - Ulepszenia wydajności, wsparcie dla Web API oraz poprawa obsługi SignalR.
2016	Premiera ASP.NET Core 1.0 - Nowa, wieloplatformowa wersja ASP.NET, przeznaczona do budowy nowoczesnych aplikacji webowych.
2017	ASP.NET Core 2.0 - Wprowadzenie nowych funkcji, takich jak Razor Pages oraz ulepszenia wydajności.
2019	ASP.NET Core 3.0 - Dodanie wsparcia dla aplikacji desktopowych (Windows Forms, WPF) oraz Blazor.
2020	ASP.NET 5.0 - Kontynuacja rozwoju z poprawkami i nowymi funkcjami w ramach .NET 5.
2021	ASP.NET 6.0 - Ulepszenia w zakresie wydajności oraz nowe funkcje, takie jak Minimal APIs; z
2022	ASP.NET 7.0 - ulepszenia i nowe funkcje, które mają na celu poprawę wydajności, łatwości użycia i elastyczności w tworzeniu nowoczesnych aplikacji webowych (np. zmiany w Minimal APIs, Razor Pages)
2023	ASP.NET 8.0 - poprawa stabilności, usprawnienie biblioteki odpowiedzialnych za serializację danych,...

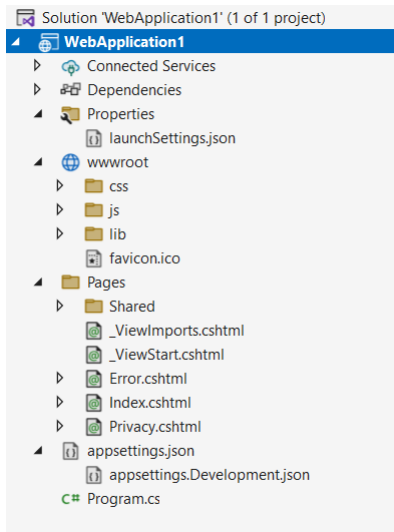
9

Podstawowe komponenty .NET (w kontekście aplikacji WWW)



10

Struktura projektu Razor Pages w ASP.NET



wwwroot/ - Katalog na pliki statyczne, takie jak CSS, JavaScript, obrazy itp.

css/: pliki stylów CSS.

js/: pliki JavaScript.

lib/: zewnętrzne biblioteki, np. Bootstrap lub jQuery.

Pages/ - Główne miejsce przechowywania stron Razor Pages.

Każda strona składa się z dwóch plików:

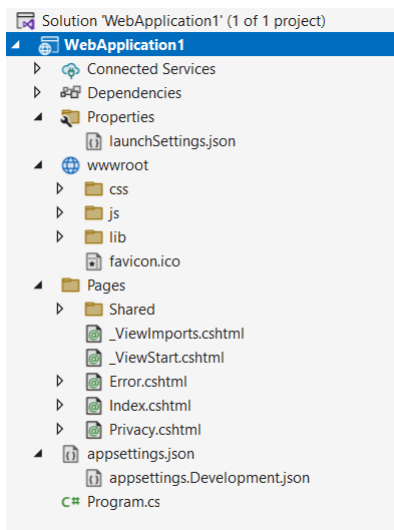
cshtml: plik Razor zawierający HTML z logiką Razor do renderowania strony.

cshtml.cs: powiązany plik C#, zawierający logikę backendową (PageModel) z metodami takimi jak OnGet(), OnPost().

Properties/ - Zawiera plik **launchSettings.json**, definiujący konfigurację uruchamiania aplikacji (np. profile debugowania).

11

Struktura projektu Razor Pages w ASP.NET

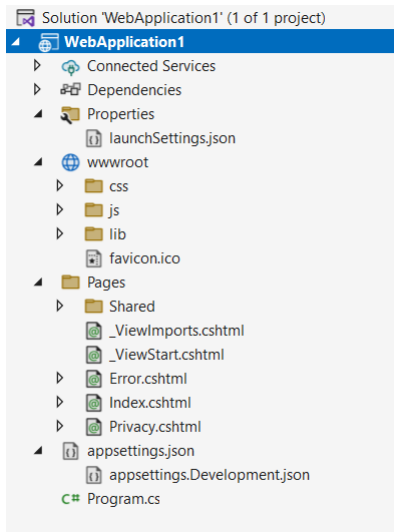


Program.cs - punkt wejścia aplikacji. W tym pliku definiowane jest uruchomienie serwera i konfiguracja usług.

Startup.cs (opcjonalnie w starszych wersjach) - w projektach Razor Pages do .NET 5 i starszych, plik ten odpowiada za konfigurację usług i middleware. W nowszych wersjach (od .NET 6) część jego funkcji przeniesiono do **Program.cs**.

12

Struktura projektu Razor Pages w ASP.NET



- **Prosta struktura:** Razor Pages używa uproszczonej struktury w porównaniu do klasycznych aplikacji MVC. Zamiast kontrolerów i akcji, każda strona ma swój PageModel.
- **PageModel:** Każda strona .cshtml ma powiązany plik .cshtml.cs, który działa jako model strony, zawierając metody do obsługi żądań (np. GET, POST).
- **Separation of Concerns:** Logika aplikacji (C#) jest oddzielona od warstwy prezentacji (HTML), co wspiera zasadę rozdzielania odpowiedzialności.

13

Proces uruchamiania aplikacji ASP.NET

Program.cs: Punkt wejścia aplikacji. Tworzenie hosta (WebHost) i konfiguracja serwera.

- Host.CreateDefaultBuilder() – Tworzy domyślny host.
- ConfigureWebHostDefaults() – Konfiguruje serwer, ścieżki, i uruchamia aplikację.

Startup.cs (opcjonalne w nowoczesnym podejściu):

- ConfigureServices() – Konfiguracja usług (np. Dependency Injection, Middleware).
- Configure() – Definiowanie potoku żądań HTTP.

14

Middleware: Komponenty obsługujące żądania i odpowiedzi HTTP

Middleware w ASP.NET Core to komponenty oprogramowania, które pośredniczą w przetwarzaniu żądań HTTP i odpowiedzi w aplikacji webowej. Są to małe, niezależne jednostki, które można z łatwością dodawać do potoku przetwarzania, co umożliwia elastyczne zarządzanie cyklem życia żądań i odpowiedzi. Middleware działa w kolejności, w jakiej zostało zarejestrowane, co pozwala na swobodne komponowanie ich funkcji.

Dependency Injection: Mechanizm wstrzykiwania zależności do serwisów

Dependency Injection (DI) to wzorec projektowy, który umożliwia wstrzykiwanie zależności do obiektów zamiast ich tworzenia wewnątrz tych obiektów. Dzięki temu aplikacje są bardziej elastyczne, łatwiejsze do testowania i utrzymania. W kontekście ASP.NET Core DI jest wbudowane i stanowi kluczowy element architektury aplikacji.

Routing: System obsługi tras URL do odpowiednich stron/kontrolerów

Routing w kontekście aplikacji webowych, w tym w ASP.NET Core, to proces, który odpowiada za mapowanie żądań HTTP do odpowiednich zasobów, takich jak kontrolery, akcje lub strony. Dzięki routingowi aplikacja może odpowiadać na różne adresy URL i kierować je do odpowiednich metod, które obsługują te żądania.